Strands Agents SDK

Generative AI

Module 3 – Lesson 1

Today's activities



- Review of module 2
- Introduction to Strands Agents SDK
- Strands components
- Memory

Review: Core dimensions of responsible AI

Controllability

Having mechanisms to monitor and steer AI system behavior

Privacy & Security

Appropriately obtaining, using and protecting data and models

Safety

Preventing harmful system output and misuse

Fairness

Considering impacts on different groups of stakeholders

Veracity & Robustness

Achieving correct system outputs, even with unexpected or adversarial inputs

Explainability

Understanding and evaluating outputs generated by an AI system

Transparency

Enabling stakeholders to make informed choices about their engagement with an AI system

Governance

Incorporating best practices into the AI supply chain, including providers and deployers

Strands Agents

Strands Agents

- An open-source SDK for building and deploying AI agents
- Model-driven approach: the model is the orchestrator
  - Simple three-component architecture: model, tools, agent loop
  - Python-native and easy to get started
  - Built-in support for multiple model providers

Strands Use Cases

- Text generation
- Summarization
- Conversational agents
- Code implementation/understanding
- Retrieval augmented generation (RAG)
- Autonomous multi-step agents

- **And so much more!**

Why Strands? Model-driven, not hard-coded!

- Let the model decide how to solve the task instead of hard-coding flows
- The LLM reasons about which tools to use and in what order
- No rigid chains or predefined sequences
- Dynamic orchestration adapts to the task at hand
- Simple agent loop: think → act → observe → repeat

Why Strands? Standard interface

- Unified interface across model providers
  - Amazon Bedrock, OpenAI, Anthropic, and more
- Switch providers with a single configuration change
- Consistent tool interface
- Standard Python functions become agent tools
- Modular and extensible framework

Why Strands? Orchestration

- AI applications are becoming more complex!

- Agent loop handles multi-step reasoning automatically
- Tools are selected and executed dynamically by the model
- State is maintained across the agent's reasoning steps
- Human-in-the-loop patterns supported natively
- Accomplish diverse tasks with a single agent

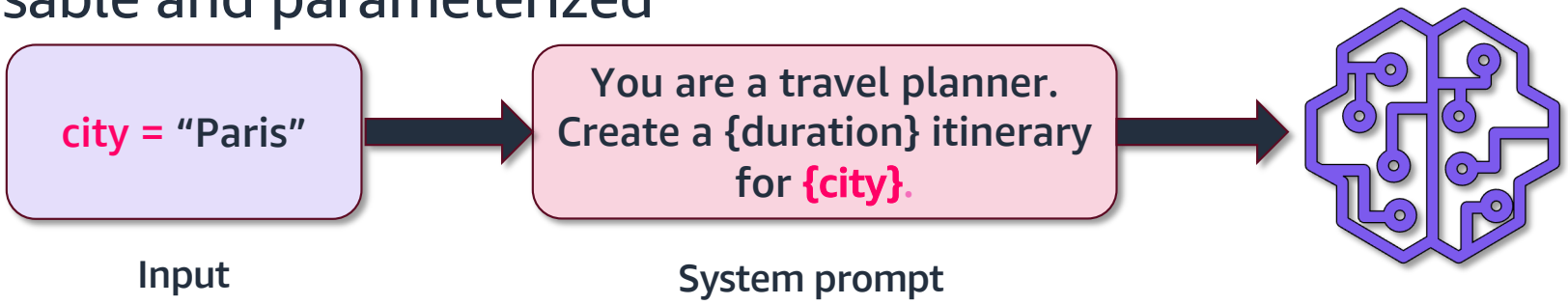
Why Strands? Observability and evaluation

- **Built-in tracing** to observe agent behavior at every step
- Test individual tools without running the full agent
- Metrics for measuring and achieving desired outcomes
  - Latency
  - Performance
  - Cost

Strands Components

System prompts

- Define the agent's behavior and personality through system prompts
- System prompts may include instructions, context, persona, constraints
- Model agnostic
- Reusable and parameterized



Structured output

- Extract structured data from agent responses
  - **JSON output via tool use**
  - **Pydantic model validation**
  - **Custom output parsing**

- Useful when the agent's output needs to adhere to specific formats for downstream tasks

Agents & Tools

- Agents use tools to interact with the world and accomplish tasks
- The model decides which tools to call and when
- Tools are simple Python functions decorated with @tool
- The agent loop orchestrates the **think → act → observe** cycle
- Easiest to get started: just define tools and create an Agent

Agent configuration

- Easily configure agent behavior

- Several options
  - Model provider selection (Bedrock, OpenAI, etc.)
  - System prompt customization
  - Tool binding and configuration
  - Max turns and stop conditions
  - Callback handlers for custom logic

Simple Agent

Create an agent with tools and a system prompt

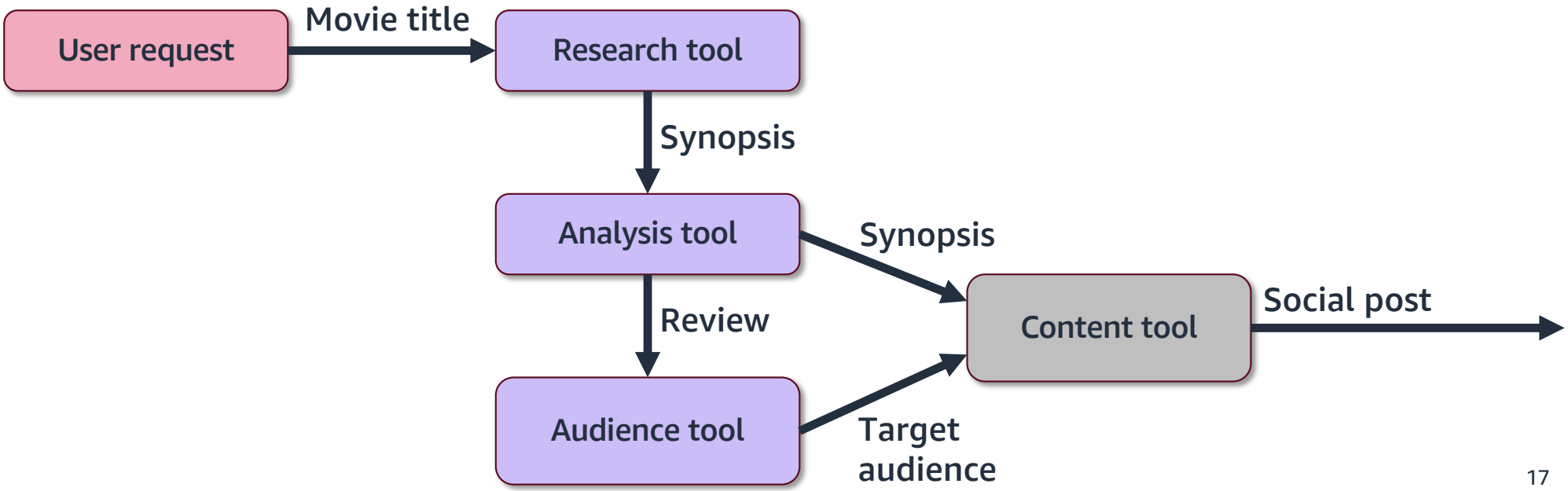
```
# Define a tool
@tool
def get_weather(location: str) -> str:
        """Get weather for a location."""
        return f"Weather in {location}: 72°F, sunny"

# Create agent
agent = Agent(
        model="us.anthropic.claude-3-5-sonnet-20241022-v2:0",
        tools=[get_weather]
)
result = agent("What's the weather in Paris?")

>> The weather in Paris is currently 72°F and sunny.
```

Connecting multiple tools

- Give the agent multiple tools and let the model orchestrate them
  - The model decides the order of tool calls based on the task

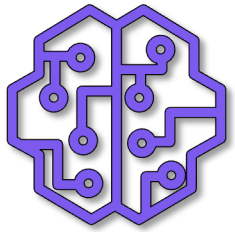


Memory

Simple prompting

What seems to be the problem?

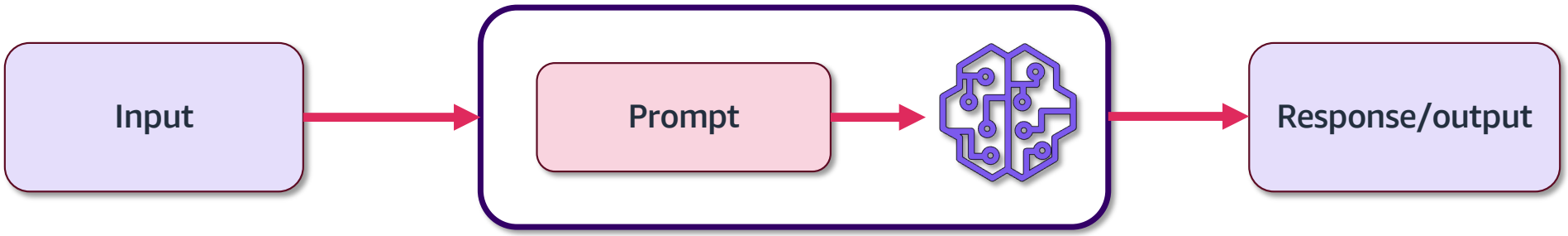




Prompt 1	Hi, my name is Andy	Hi Andy, it is nice to meet you.
Prompt 2	What is my name?	I don't know. What do I call you?

Simple agent application

- Without any component persisting context, the agent is unaware of prior interactions
- Agents are stateless by default

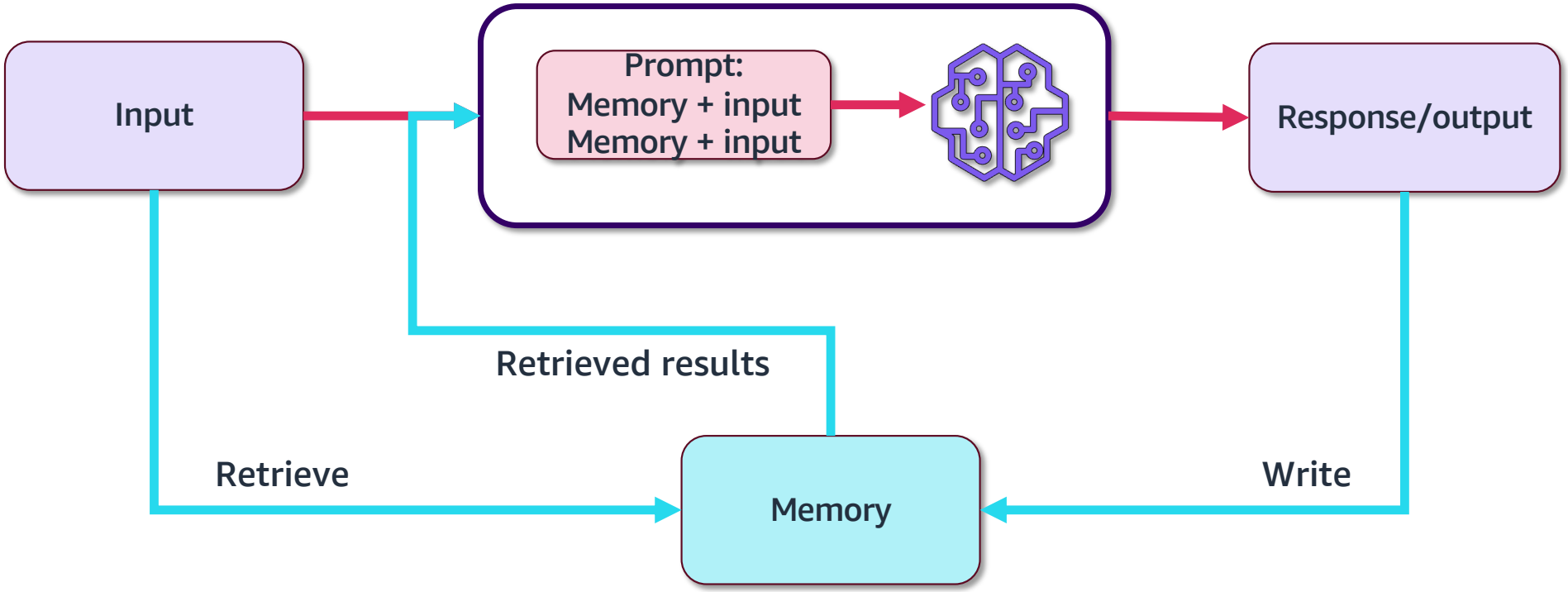


Memory

- Maintaining information that the agent can refer to
  - Remembering **past interactions**
  - Persisting **context, roles, rules**, and so on
- Allows the agent to dynamically store and access information

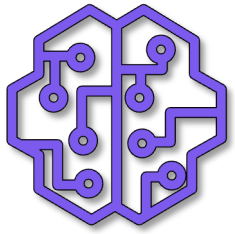
- Commonly used for chatbots and other conversational applications

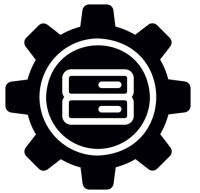
Agent with memory



Prompting with memory







Memory

Hi, my name is Andy	Hi Andy, it is nice to meet you.	Human: Hi, my name is Andy. AI: Hi Andy, it is nice to meet you.
What is my name?	You introduced yourself as Andy.	Human: Hi, my name is Andy. AI: Hi Andy, it is nice to meet you. Human: What is my name? AI: You introduced yourself as Andy

Conversational Memory

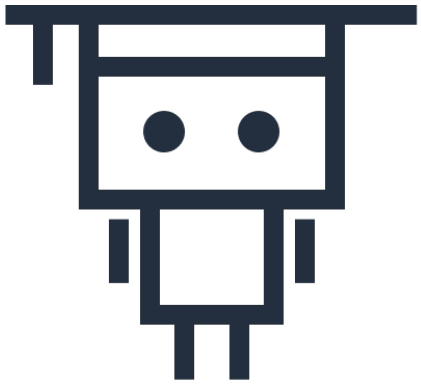
- Storing all interactions and retrieving them as history
  - **Perfect recall**

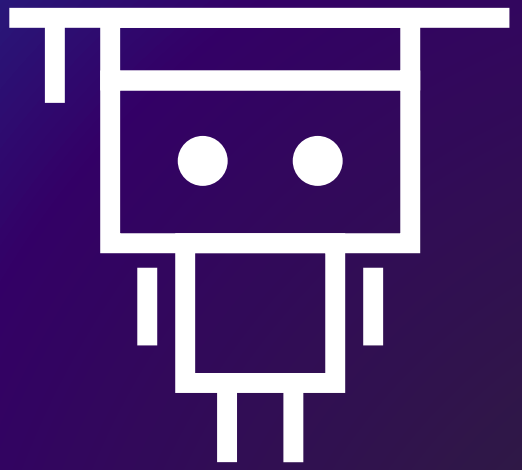
```
# Create an agent
agent = Agent(
       model="us.anthropic.claude-3-5-sonnet-20241022-v2:0",
       tools=[get_weather]
)
# Conversation with memory
agent("Hi, my name is Andy")
>> Nice to meet you, Andy!
agent("What is my name?"
>> Your name is Andy.
```

- What if the conversation is too long?
  - **We will address this in the next lesson!**

Next lesson

- This lesson covered Strands Agents SDK and its core components.
- In the next lesson, you will learn about developing conversational applications with Strands agents.





Thank you!